

Introduction

keecas is a module for performing symbolic and units-aware calculations. It has been developed to be used mainly in a jupyter notebook's interactive environments, which then could be rendered as pdf by [quarto](#).

keecas leverage some well known python modules, wrapping some of their functions with sensisble defaults, aiming to reduce boiler plate code and be quick to use. Some of the modules used are:

sympy for all symbolic expression and computation

pint convenient unit registry; **pint** quantities can be converted in `sympy.physics.units` directly

pipe some of the most used **sympy** functions are wrapped in **pipe** object, so that they can be used in a sequential fashion

Quick Start

Let's start with a bare minimum example: let's calculate the maximum bending moment (M_{Sd}) for a beam of span l , simply supported at the end (pinned), uniformly loaded (q).

```
# import everything from keecas
from keecas import symbols, u, pc, show_eqn
from keecas import *

# some notable imports:
# - `symbols`: function to define symbols from sympy module
# - `u`: unit registry from pint module
# - `pc`: short for `keecas.pipe_command` where are the most used sympy function are defined

config.col_wrap[1][object] = ('=', '')
```

```

# define the symbols with symbols() from `sympy`
q, l, M_Sd = symbols(r'q, l, M_{Sd}')

# define the parameters with units support provided by `pint` (`u` is the unit registry)
_p = {
    q: S(5*u('kN/m')),
    l: S(400*u.cm),
}

# define the symbolic expression
_e = {
    M_Sd: "q*l^2/8" # the expression is written as a simple string
    | pc.parse_expr # function that will parse the expression (from `sympy`)
}

# evaluate the expression in `_e`
_v = {
    k: v
    | pc.subs(_e|_p) # substitute the symbol contained in the joined dict _e|_p (from `sympy`)
    | pc.convert_to([u.kN, u.m]) # convert the expression (from `sympy`) to the preferred units
    | pc.N # evaluate the expression as decimal (from `sympy`)
    for k,v in _e.items() # get all the expression contained in _e
}

# add some description
_d = {
    M_Sd: "max bending moment",
    q: "uniform load",
    l: "span of the beam",
}

# create the latex expression (amsmath) to be displayed as Markdown object
show_eqn(
    [_p|_e, _v, _d], # list of dict to be shown
    float_format='{:.2f}', # precision of floats
    # col_wrap=[ # columns wrappers
    #     None, # wrapping for the 1st column (key of first dict in the main argument list)
    #     '=', # add '=' sign before the 2nd column (value of first dict in the main argument)
    #     '=', # add '=' sign before the 3rd column (value of second dict in the main argument)
    #     (r'\quad(', '))' # wrap the 4th column with '\quad(' on the left and ')' on the right
    # ]
)

```

$$q = \frac{5.00 \text{ kN}}{\text{m}} \quad \text{uniform load} \quad (1)$$

$$l = 400 \text{ cm} \quad \text{span of the beam} \quad (2)$$

$$M_{Sd} = \frac{ql^2}{8} = 10.00 \text{ kN m} \quad \text{max bending moment} \quad (3)$$

Explanation

What is a math equation if not a mapping between the left hand side (LHS) and the right hand side (RHS) of the equal sign? Therefore the simplest and easy to write python container we can write is a `dict` where the LHS is the key, and RHS is the value.

In the above example we have provided 3 different `dict`:

`_p` dict containing all the parameters

`_e` dict containing all the symbolic, unevaluated, expression

`_v` dict containing the evaluated expression, written as a dict comprehension of the `k, v` pairs of `_e`

All the dict are then combined in a list to be used of `show_eqn`, the function that will create a latex block (amsmath) to be displayed as a Markdown object by the jupyter notebook (KaTeX in case of vscode)

The main argument of the `show_eqn` function can be a `dict` or a `list[dict]`. The key, value pairs of the first dict will be displayed as 1st and 2nd columns of the latex amsmath block, while only the value of the subsequent dict will be shown in the following columns if there is a key match (note that `_v` does not contain neither `q` or `l` keys, so nothing is shown in the output for that 3rd column.)

We can have a look at the latex code by passing `debug=True` to the function or setting globally `options.DEBUG=True`

```
# options.DEBUG = True

show_eqn(
    [_p|_e, _v, _d], # list of dict to be shown
    float_format='{:.2f}', # precision of floats
    # col_wrap=[ # columns wrappers
    #     None, # wrapping for the 1st column (key of first dict in the main argument list ->
    #     '=', # add '=' sign before the 2nd column (value of first dict in the main argument
    #     '=', # add '=' sign before the 3rd column (value of second dict in the main argumen
```

```
# (r'\quad(', ')') # wrap the 4th column with '\quad(' on the left and ')' on the right
# ],
debug=True,
)
```

$$q = \frac{5.00 \text{ kN}}{\text{m}} \quad \text{uniform load} \quad (4)$$

$$M_{Sd} = \frac{ql^2}{8} = 10.00 \text{ kN m} \quad \text{max bending moment} \quad (6)$$

In the previous example all the `dict` were prepended by `_`: that because those dict are not really meant to be preserved further than the cell they are defined. They are meant to only collect the expressions/parameters of the cell to be displayed, and then discarded. To preserve all the expression/parameters of the notebook we will initialize a named dict.

We will calculate the notable values for a simple supported beam with a uniform load in ULS condition for bending and shear, and SLS condition for deflection.

```

l, b_i = symbols(r"l, b_{i}")

_p = {
    l: 6*u.m,
    b_i: 3*u.m,
}
params.update(_p) # save the parameters in the notebook dict

_d = {
    l: "span of the beam",
    b_i: "width of influence",
}

show_eqn([_p, _d], col_wrap=[None, "=", "&"], environment="cases")

```

$$\begin{cases} l = 6.00 \text{ m} & \text{span of the beam} \\ b_i = 3.00 \text{ m} & \text{width of influence} \end{cases} \quad (7)$$

Actions

Let's calculate the load applied to a simple beam.

Loads

```

G_1, G_2, Q_k = symbols(r"G_{1}, G_{2}, Q_{k}")

# define the loads in whatever units you want
_p = {
    G_1: 2.5 * u.kPa,
    G_2: 300 * u("daN/m^2"),
    Q_k: 4 * u.kN / u.m**2,
}
params.update(_p) # save the parameters in the notebook dict

_d = {
    G_1: "permanent loads",
    G_2: "permanent non structural",
}

```

```

    Q_k: "live loads",
}

# NOTE: values will be rendered as `pint` quantities because they are not being converted yet
show_eqn([_p, _d], col_wrap=[None, "=", "&"])

```

$$G_1 = 2.50 \text{ kPa} \quad \text{permanent loads} \quad (8)$$

$$G_2 = 300.00 \text{ decanewton/m}^2 \quad \text{permanent non structural} \quad (9)$$

$$Q_k = 4.00 \text{ kilonewton/m}^2 \quad \text{live loads} \quad (10)$$

Safety coefficients for loads

```

gamma_G1, gamma_G2, gamma_Qk = symbols(r'\gamma_{G_{1}}', \gamma_{G_{2}}' , \gamma_{Q_{k}}')

_p = {
    gamma_G1: 1.3,
    gamma_G2: 1.5,
    gamma_Qk: 1.5,
}
params.update(_p)

# since _p is being redefined in this cell, it will only display the content of this cell
show_eqn(_p)

```

$$\gamma_{G_1} = 1.3 \quad (11)$$

$$\gamma_{G_2} = 1.5 \quad (12)$$

$$\gamma_{Q_k} = 1.5 \quad (13)$$

Applied forces

The applied load are multiplied for the width of influence b_i of the beam:

```

F_d, F_k = symbols(r"F_{d}, F_{k}")

_e = {
    F_k: "(G_1 + G_2 + Q_k)*b_i" | pc.parse_expr,
    F_d: "(gamma_G1*G_1+gamma_G2*G_2+gamma_Qk*Q_k)*b_i" | pc.parse_expr,
}
eqn.update(_e) # save the expressions in the notebook dict

_v = {
    k: (
        v
        | pc.subs(eqn | params)
        | pc.convert_to([u.kN, u.m])
        | pc.N
        | pc.as_two_terms(as_mul=True) # this will display nicely the values and units in the
    )
    for k, v in _e.items()
}

_d = {
    F_k: "(SLS)",
    F_d: "(ULS)",
}

show_eqn([_e, _v, _d], col_wrap=[None, "=", "=", "&"], float_format='{:.2f}')

```

$$F_k = (G_1 + G_2 + Q_k) b_i = 28.50 \frac{\text{kN}}{\text{m}} \quad (\text{SLS}) \quad (14)$$

$$F_d = (\gamma_{G_1} G_1 + \gamma_{G_2} G_2 + \gamma_{Q_k} Q_k) b_i = 41.25 \frac{\text{kN}}{\text{m}} \quad (\text{ULS}) \quad (15)$$

ULS: Bending Moment and Shear

We calculate the bending moment and shear in ULS condition:

```

M_Sd, V_Sd = symbols(r"M_{Sd}, V_{Sd}")

_e = {
    M_Sd: "F_d * l^2 / 8" | pc.parse_expr,

```

```

    V_Sd: "F_d * l / 2" | pc.parse_expr,
}
eqn.update(_e) # save the expressions in the notebook dict

_v = {
    k: (
        v
        | pc.subs(eqn | params)
        | pc.convert_to([u.kN, u.m])
        | pc.N
        | pc.as_two_terms(as_mul=True) # this will display nicely the values and units in the
    )
    for k, v in _e.items()
}

# we can be finer with the float format options
_f = {
    M_Sd: '{:.3f}',
    V_Sd: '{:.0f}',
}

show_eqn([_e, _v, _d], col_wrap=[None, "=", "=", "&"], float_format=_f)

```

$$M_{Sd} = \frac{F_d l^2}{8} = 185.625 \text{ kN m} \quad (16)$$

$$V_{Sd} = \frac{F_d l}{2} = 124 \text{ kN} \quad (17)$$

SLS: deflection

We calculate the deflection of the beam with the following equation:

```

f, E, J = symbols(r"f , E, J")

# since we want to collect display the expression as two separate terms (5/384) and (F_k * l^4 / (E * J))
_e = {
    # f : " (5/384) * (F_k * l^4 / (E * J))" | pc.parse_expr,
    f : " S(5/384) * S(F_k * l^4 / (E * J))" | pc.parse_expr,
    # f : " N(5/384) * S(F_k * l^4 / (E * J))" | pc.parse_expr, # th N() will evaluate the fr

```

```

}
eqn.update(_e)

show_eqn(_e)

```

$$f = \frac{5}{384} \frac{F_k l^4}{E J} \quad (18)$$

Assuming a IPE270 beam, the deflection results

```

_p = {
    E: 210000 * u("MPa"),
    J: 8356*u.cm**4
}
params.update(_p)

_v = {
    k: (
        v
        | pc.subs(eqn | params)
        | pc.convert_to([u.kN, u.mm])
        | pc.N
    )
    for k, v in _e.items()
}

# I evaluate a simple expression that has not its own symbol defined
__v = {
    k: k | pc.subs(eqn | params) | pc.convert_to([u.kN, u.mm]) | pc.N for k in [1/f]
}

_d = {
    1/f: "ratio span of the beam / deflection",
}

show_eqn([_p|_v|__v, _d], col_wrap=[None, "="], float_format='{:.2f}')

```

$$E = 210000.00 \text{ MPa} \quad (19)$$

$$J = 8356.00 \text{ cm} \quad (20)$$

$$f = 27.41 \text{ mm} \quad (21)$$

$$\frac{l}{f} = 218.92 \quad = \text{ratio span of the beam / deflection} \quad (22)$$

```
config.language = "it"
show_eqn(
  {
    'verifica' : verifica(1, 1)
  }
)
```

$$\text{verifica} \quad [\leq 1 \quad \text{VERIFICATO}] \quad (23)$$

```
show_eqn(
  {
    'piecewise': '''Piecewise(
      (1, x<0),
      (2, True),
    )
    ''' | pc.parse_expr
  }
)
```

$$\text{piecewise} = \begin{cases} 1 & \text{per } x < 0 \\ 2 & \text{altrimenti} \end{cases} \quad (24)$$